JPA & Hibernate

Java Backend Interview — Short Notes

1. JPA vs Hibernate Overview

	JPA	Hibernate
Type	Specification (JSR 338) — interface/contract	Implementation of JPA + extra features
Package	jakarta.persistence.*	org.hibernate.*
Portability	Vendor-neutral — switch implementations	Hibernate-specific — tied to Hibernate
Features	Core ORM — entities, queries, transactions	JPA features + caching (L2), HQL, Envers, spatial
Usage	Code to JPA interfaces — more portable	Access Hibernate-specific features via Session API
Config	persistence.xml or Spring auto-config	hibernate.cfg.xml or Spring properties

2. Entity Mapping

2.1 Core Entity Annotations

Annotation	Purpose	Notes
@Entity	Mark class as JPA entity — mapped to DB table	Must have @Id. Class must be non-final
@Table	Customize table name, schema, indexes, uniqueConstraints	@Table(name="orders",indexes=@Index(...))
@Id	Primary key field	Required. Must be serializable
@GeneratedValue	PK generation strategy	AUTO, IDENTITY, SEQUENCE, TABLE. Prefer SEQUENCE for performance
@Column	Customize column — name, nullable, length, unique, updatable	@Column(name="email",nullable=false,unique=true)
@Transient	Exclude field from persistence — not mapped to column	In-memory computed fields
@Enumerated	Map enum — ORDINAL (int, brittle) or STRING (safe)	Always use STRING — ORDINAL breaks on enum reorder
@Temporal	Map Date/Calendar — DATE, TIME, TIMESTAMP	Prefer LocalDate/LocalDateTime — no @Temporal needed
@Lob	Large object — CLOB (text) or BLOB (binary)	Use for large text or file storage
@Embedded/@Embeddable	Embed value object into entity's table	Address embedded in User — same table, no join
@Version	Optimistic locking — auto-incremented on each update	Throws OptimisticLockException on concurrent update
@CreationTimestamp	Auto-set on insert (Hibernate-specific)	@CreatedDate (Spring Data) for JPA standard

@UpdateTimestamp	Auto-set on update (Hibernate-specific)	@LastModifiedDate (Spring Data) for JPA standard

```
@Entity @Table(name="orders")
public class Order {
@Id @GeneratedValue(strategy=GenerationType.SEQUENCE,
generator="order_seq")
@SequenceGenerator(name="order_seq",sequenceName="order_seq",allocationSize=50)
private Long id;
@Column(nullable=false,length=50)
private String status;
@Enumerated(EnumType.STRING)
private OrderType type;
@Version private Long version; // optimistic locking
@CreationTimestamp private LocalDateTime createdAt;
}
```

2.2 Relationships

Annotation	Cardinality	Default Fetch	Table Structure	Notes
@OneToOne	1:1	EAGER	FK in owning entity or @JoinColumn	Use LAZY unless always needed
@OneToMany	1:N	LAZY	FK in child table or join table	Owning side usually on Many side
@ManyToOne	N:1	EAGER	FK in entity with @ManyToOne	Most common — avoid EAGER
@ManyToMany	N:M	LAZY	Join table with two FKs	Use @JoinTable. Often model explicit join entity

```
// OneToMany — bidirectional
@Entity public class Order {
@OneToMany(mappedBy="order", cascade=CascadeType.ALL, orphanRemoval=true)
private List items = new ArrayList<>();
}
@Entity public class OrderItem {
@ManyToOne(fetch=FetchType.LAZY)
@JoinColumn(name="order_id")
private Order order;
}
```

■ EAGER fetch on @ManyToOne loads related entity on every query — N+1 risk. Always LAZY, join when needed

2.3 Cascade Types

Cascade Type	Propagates
PERSIST	save() propagates to related entities
MERGE	update() propagates to related entities
REMOVE	delete() propagates — delete parent deletes children
REFRESH	refresh() propagates — reload from DB
DETACH	detach() propagates — remove from persistence context
ALL	All above. Common for parent-child (Order → OrderItems)

• **orphanRemoval=true** — remove child entity when removed from parent collection. Different from REMOVE cascade

3. Entity Lifecycle & Persistence Context

3.1 Entity States

State	Description	Tracked by PC?	Transition
Transient	New object — not associated with DB, not in PC	No	new Order() — just a Java object
Managed	Associated with PC — changes auto-persisted on flush	Yes — dirty checked	em.persist() or loaded by em.find()/query
Detached	Was managed, now disconnected from PC — changes not tracked	No	em.detach(), em.close(), serialized/deserialized
Removed	Marked for deletion — DELETE on next flush	Yes	em.remove() on managed entity

3.2 Persistence Context & Session

• **Persistence Context (PC)** — first-level cache. Tracks managed entities. Scoped to transaction (default)

• Within same PC: em.find() twice for same ID returns SAME object (== comparison). Guarantees no duplicates

• **Flush** — sync PC changes to DB (SQL generated). Flush modes: AUTO (before query), COMMIT (on transaction commit), MANUAL

• **Dirty checking** — Hibernate tracks original entity state. On flush, generates UPDATE for changed fields only

```
// Dirty checking — no explicit save needed
@Transactional
public void updateUserName(Long id, String name) {
User user = userRepo.findById(id).get(); // managed
user.setName(name); // no save() needed — dirty check on flush
} // transaction commit → flush → UPDATE user SET name=? WHERE id=?
```

4. N+1 Problem & Solutions

• Fetching N entities, then making 1 additional query per entity for related data = N+1 queries total

```
// N+1 PROBLEM
List orders = orderRepo.findAll(); // 1 query — SELECT * FROM orders
for(Order o : orders) {
o.getItems().size(); // N queries — 1 per order — LAZY load triggered
}
```

Solutions:

• **JOIN FETCH (JPQL)** — single query with JOIN FETCH — loads orders + items together

```
@Query("SELECT DISTINCT o FROM Order o JOIN FETCH o.items WHERE o.id IN :ids")
List findWithItems(@Param("ids") List ids);
```

• **@EntityGraph** — specify which associations to eagerly load for specific query

```
@EntityGraph(attributePaths={"items","items.product"})
Optional findWithItemsById(Long id);
```

• **@BatchSize** — Hibernate loads N lazy associations in one IN query instead of N queries

```
@OneToMany @BatchSize(size=25)
private List items;
```

• **FetchMode.SUBSELECT** — one subselect query for all lazy collections of all parent instances

```
@Fetch(FetchMode.SUBSELECT)
private List items;
```

✔ Always check generated SQL in dev: spring.jpa.show-sql=true. Use Hibernate statistics or p6spy to catch N+1 in tests

5. Caching

Cache Level	Scope	Description	Config
L1 — Persistence Context	Single session/transaction	Built-in, automatic. Same entity found twice returns same object from cache	Always on. em.clear() to invalidate
L2 — Shared Cache	Application-wide (all sessions)	Optional. Ehcache, Caffeine, Hazelcast, Redis. Cache entities, queries, collections	@Cache + spring.jpa.properties.hibernate.cache.use_second_level_cache=true
Query Cache	Application-wide	Caches JPQL query results. Only useful with L2. Cache invalidated on any entity update in table	hibernate.cache.use_query_cache=true + @QueryHints

```
@Entity @Cache(usage=CacheConcurrencyStrategy.READ_WRITE)
public class Product { // L2 cached entity
@OneToMany @Cache(usage=CacheConcurrencyStrategy.READ_WRITE)
private List categories; // L2 cache for collection
}
```

L2 Strategy	Description	Use When
READ_ONLY	No updates allowed — pure read cache. Fastest	Reference/static data (countries, currencies)
NONSTRICT_READ_WRITE	No strict lock. Eventual consistency — brief inconsistency possible	Rarely updated data where brief staleness OK
READ_WRITE	Soft locks on update — consistent. Overhead	Updated data needing consistency
TRANSACTIONAL	Full XA transaction aware. Strongest. JTA required	JTA environments needing full consistency

6. Transactions & Locking

6.1 Optimistic vs Pessimistic Locking

	Optimistic	Pessimistic
Mechanism	@Version column — compare on update. No DB lock	DB-level lock — SELECT FOR UPDATE
On conflict	OptimisticLockException — retry or fail	Other transactions wait or timeout
Performance	Better — no blocking	Worse under contention — blocking
Use when	Low contention — most reads, occasional writes	High contention — competing writes, critical sections
LockModeType	OPTIMISTIC, OPTIMISTIC_FORCE_INCREMENT	PESSIMISTIC_READ, PESSIMISTIC_WRITE, PESSIMISTIC_FORCE_INCREMENT

```
// Pessimistic write lock
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT p FROM Product p WHERE p.id=:id")
Optional findByIdForUpdate(@Param("id") Long id);
```

6.2 Transaction Isolation Levels

Level	Dirty Read	Non-Repeatable	Phantom Read	Description
READ_UNCOMMITTED	Yes	Yes	Yes	Can read uncommitted changes. Fastest, dangerous
READ_COMMITTED	No	Yes	Yes	Default in most DBs (PostgreSQL, Oracle). Only committed data read
REPEATABLE_READ	No	No	Yes	Default MySQL InnoDB. Same row reads always consistent
SERIALIZABLE	No	No	No	Fully isolated. Slowest. Full sequential execution

7. Spring Data JPA

7.1 Repository Hierarchy

Interface	Methods	Notes
Repository	Marker only — no methods	Base marker interface
CrudRepository	save, findById, findAll, delete, count, existsById	Basic CRUD
PagingAndSortingRepository	findAll(Pageable), findAll(Sort)	Adds pagination and sorting
JpaRepository	All above + flush, saveAndFlush, deleteInBatch, getReferenceById	JPA-specific extras. Use this one
JpaSpecificationExecutor	findAll(Specification), count(Specification)	Dynamic queries with Specification/Criteria API

7.2 Derived Query Methods

Keyword	SQL	Example
findBy	SELECT WHERE	findByEmail(String email)
findBy...And	WHERE x=? AND y=?	findByFirstNameAndLastName(String f, String l)
findBy...Or	WHERE x=? OR y=?	findByEmailOrUsername(String e, String u)
findBy...Like	WHERE x LIKE ?	findByNameLike(String pattern)
findBy...Containing	WHERE x LIKE '%?%'	findByNameContaining(String part)
findBy...StartingWith	WHERE x LIKE '?%'	findByEmailStartingWith(String prefix)
findBy...Between	WHERE x BETWEEN ? AND ?	findByCreatedAtBetween(LocalDate from, LocalDate to)
findBy...GreaterThan	WHERE x > ?	findByAgeGreaterThan(int age)
findBy...In	WHERE x IN (?)	findByStatusIn(List statuses)
findBy...IsNull	WHERE x IS NULL	findByDeletedAtIsNull()
findBy...OrderBy	ORDER BY	findByStatusOrderByCreatedAtDesc(String s)
countBy...	SELECT COUNT(*)	countByStatus(String status)
existsBy...	SELECT 1 WHERE EXISTS	existsByEmail(String email)
deleteBy...	DELETE WHERE	deleteByStatus(String status)

7.3 @Query & Projections

```
// JPQL query
@Query("SELECT o FROM Order o WHERE o.status=:status AND o.total>:min")
List findByStatusAndTotalGreaterThan(@Param("status") String s,
@Param("min") BigDecimal min);
// Native SQL query
@Query(value="SELECT * FROM orders WHERE created_at>NOW()-INTERVAL '7 days'",
nativeQuery=true)
List findRecentOrders();
// Modifying query
@Modifying @Transactional
@Query("UPDATE Order o SET o.status=:status WHERE o.id IN :ids")
int updateStatus(@Param("status") String s, @Param("ids") List ids);
// Interface-based projection — fetch only needed columns
public interface OrderSummary {
Long getId(); String getStatus(); BigDecimal getTotal();
}
List findByStatus(String status); // SELECT id,status,total only
```

8. Performance Best Practices

- **Use LAZY fetch by default** — @ManyToOne and @OneToOne should be LAZY unless always needed
- **Prefer DTO projections** over full entity for read-only use cases — avoids loading unnecessary data
- **Bulk operations** — @Modifying @Query for mass updates/deletes instead of loading all entities
- **Pagination** — never fetch unbounded collections. Use Pageable everywhere
- **Enable batch inserts** — spring.jpa.properties.hibernate.jdbc.batch_size=50
- **Use SEQUENCE generator** — IDENTITY (auto-increment) disables batch inserts. SEQUENCE doesn't
- **StatelessSession** — Hibernate-specific. No L1 cache, no dirty checking. For bulk ETL operations
- **Index foreign keys** — JPA doesn't create FK indexes automatically. Add @Index on @Table
- **Avoid bidirectional with equals/hashCode on mutable fields** — use business key or UUID

9. Interview Questions

Core JPA

Q1. JPA vs Hibernate?

→ JPA is specification (interface). Hibernate is JPA implementation + extras. Code to JPA interfaces for portability. Use Hibernate-specific API only when JPA doesn't cover the use case (L2 cache strategy, batch fetch)

Q2. What is persistence context?

→ Unit of work — tracks managed entities. First-level cache. All entities loaded in same session are identity-guaranteed (same object). Flushed to DB on transaction commit. Scoped to transaction by default in Spring

Q3. Entity lifecycle states?

→ Transient: new object, not in PC. Managed: in PC, changes tracked, auto-persisted on flush. Detached: was managed, PC closed/detached — changes not tracked. Removed: scheduled for DELETE

Q4. What is dirty checking?

→ Hibernate snapshots entity state on load. Before flush, compares current state to snapshot. Generates UPDATE only for changed fields. No need to call save() in @Transactional method — dirty checking handles it

Q5. What is the N+1 problem?

→ 1 query for N parents + N queries for each child collection = N+1 total. Caused by LAZY loading in a loop. Fix: JOIN FETCH, @EntityGraph, @BatchSize. Detect with: show-sql + Hibernate statistics

Q6. EAGER vs LAZY fetch?

→ EAGER: load related entity immediately with parent. LAZY: load on first access (proxy). Default: @ManyToOne=EAGER, @OneToMany=LAZY. Always use LAZY — EAGER causes performance issues and N+1 risk

Mapping & Caching

Q7. What is @Version? How does optimistic locking work?

→ @Version on Long/Integer field. Hibernate includes WHERE version=? in UPDATE. If no rows updated (concurrent update changed version), throws OptimisticLockException. Client must retry. No DB-level lock — non-blocking

Q8. Cascade.REMOVE vs orphanRemoval?

→ Cascade.REMOVE: deleting parent also deletes children via JPA cascade. orphanRemoval=true: removing child from parent collection triggers DELETE for removed child. Use orphanRemoval for collections where child cannot exist without parent

Q9. L1 vs L2 cache?

→ L1: persistence context (per session/transaction) — built-in, automatic, scoped to one transaction. L2: application-wide shared cache (Ehcache/Caffeine). Entities shared across sessions. Must explicitly configure and annotate with @Cache

Q10. What is @MappedSuperclass?

→ Base class with common fields (id, createdAt, updatedAt) — not an entity itself. Subclasses inherit mappings. No separate table for superclass. Alternative: @Inheritance with TABLE_PER_CLASS strategy

Q11. Inheritance mapping strategies?

→ SINGLE_TABLE (default): all subclasses in one table + discriminator column. Fast queries, nullable columns. TABLE_PER_CLASS: each concrete class its own table. JOINED: each class its own table, joined on query. JOINED is normalized but slower

Spring Data JPA

Q12. save() vs saveAndFlush() in Spring Data?

→ save(): marks entity as managed (or merges detached). Persisted on transaction commit. saveAndFlush(): immediately flushes to DB within current transaction. Use saveAndFlush() when you need the ID or to see effect in same transaction

Q13. What is @Modifying?

→ Required for @Query methods that modify data (UPDATE/DELETE). Without it: throws InvalidDataAccessApiUsageException. Add @Transactional alongside. @Modifying(clearAutomatically=true) clears L1 cache after bulk update

Q14. What is a projection?

→ Fetch only required columns instead of full entity. Interface-based: Spring generates proxy. Class-based (DTO): constructor expression in @Query. Reduces data transfer and avoids loading unnecessary associations

Q15. getReferenceById() vs findById()?

→ getReferenceById(): returns proxy (no DB hit). DB queried only when property accessed. Useful for FK associations — avoid loading just to set FK. findById(): immediate SELECT. Returns Optional

Advanced

Q16. What is @Transactional(readOnly=true)?

→ Hints Hibernate to skip dirty checking at flush (MANUAL flush mode). DB may route to read replica. Improves performance for read-only queries. Always use on @Query methods that only read

Q17. How do you handle large datasets?

→ Stream with scrollable cursor. @Query returning Stream — process without loading all in memory. Or Slice for efficient pagination (no COUNT query). Or chunked processing with Pageable

Q18. What is the Open Session in View antipattern?

→ Keep Hibernate session open through view rendering — allows lazy loading in templates. Bad: DB connection held for entire HTTP request duration, performance issues, hides N+1 problems. Disable: spring.jpa.open-in-view=false

Q19. How do you do bulk inserts efficiently?

→ Enable batch: hibernate.jdbc.batch_size=50 + hibernate.order_inserts=true. Use SEQUENCE not IDENTITY generator. saveAll(list) then flush. Or use EntityManager.persist() in batches with clear() every N entities

Q20. What is Specification pattern in Spring Data?

→ Type-safe dynamic queries using JPA Criteria API. Specification interface: toPredicate(Root,CriteriaQuery,CriteriaBuilder). Composable: Specifications.where(a).and(b).or(c). Better than @Query for dynamic filter combinations